Overfitting the Judge: Co-Tuned Evaluators Turn Generate-and-Select Agents into P-Hackers

Adyuth Chirakala
Dougherty Valley High School
San Ramon, California, USA
adychirakala@gmail.com

Parv Mehndiratta
Dougherty Valley High School
San Ramon, California, USA
0103.parv@gmail.com

ABSTRACT

LLM pipelines that both *propose* candidates and *judge* them—the generate-and-select loop—are now standard in agentic search. Prior work established that selection against a *static* proxy judge inflates the reported score of the winner (best-of-n reward hacking), and that ensembles of independent reward models mitigate it. We study the case those results do not cover: a judge that is *co-tuned*—its parameters refit on the generator's own search trace, the common situation when a scoring prompt or rubric is iterated until it "looks right" on the pipeline's own outputs. In a controlled, deterministic, zero-noise testbed we run a four-condition provenance ablation: a fixed exploitable judge, a co-tuned judge, an ensemble of co-tuned judges, and an ensemble of judges never exposed to the trace, with the search structure held identical. Three results, each holding in 100% of 24 independent reseeds and, directionally, in 100% of seeds in every cell of a 12-cell parameter-sensitivity sweep (including a gentler interpolating co-tuning rule): (1) co-tuning is a Goodhart runaway—the calibration gap (reported minus true quality) of a co-tuned judge is ∼4× that of an already-exploitable static judge (211.5 vs 52.8 at N=3,000), and the co-tuned winner's *true* quality is the worst of all conditions; (2) ensembling does *not* fix it when the ensemble shares the trace—co-tuned-ensemble inflation stays ∼13× the independent ensemble's (156.9 vs 12.0); (3) *independence from the search trace*, not ensembling per se, is the active ingredient: the trace-independent ensemble both nearly closes the calibration gap and selects candidates of ∼3.8× the true quality of the co-tuned judge's picks. We position the result against best-of-n reward hacking, reward-model ensembling, herding in correlated ensembles, and LLM-as-judge reliability, and state its limits: the testbed is synthetic, and a live-LLM instantiation is the immediate next step.

KEYWORDS

LLM-as-judge, reward hacking, evaluator overfitting, generate-and-select agents, Goodhart's law, ensembles

1 INTRODUCTION

A growing share of AI pipelines follows one pattern: a generator proposes many candidate artifacts—trading strategies, code patches, proofs, prompts—and a judge scores them so the pipeline can keep the best. The loop is usually analyzed as a data problem: does the *artifact* generalize? In quantitative finance this is backtest overfitting, corrected by multiple-testing deflation [11, 12]. We ask an orthogonal question about the *judge*: what happens to the reported score when the judge itself was tuned, prompted, or selected using the same trace of candidates the generator is searching over?

This situation—call the judge *co-tuned*—is the practical default, not an edge case. A scoring prompt iterated until its rankings "look right" on the pipeline's own outputs; a rubric refined against last week's generations; an evaluation harness patched whenever the system under test finds a loophole: in each, the judge's parameters absorb information about the generator's search region, so the judge's errors become *correlated with exactly the part of candidate-space being searched*.

Prior work covers the neighboring cases. Selection against a *static* proxy judge inflates reported scores with a known scaling law—Gao et al. measure this directly for best-of-n sampling, not just RL [1]—and recent work names and formalizes inference-time reward hacking for the best-of-n class [4]. Ensembles of independent reward models mitigate best-of-n overoptimization [2], with ensemble *diversity* identified as the load-bearing property [3]. What none of these measure is the co-tuned case: a judge shaped by the generator's own trace, and whether ensembling without independence still helps.

Contributions. (1) A controlled, deterministic, zero-noise testbed in which judge *provenance* is the only manipulated variable across four conditions (fixed / co-tuned / co-tuned ensemble / trace-independent ensemble), with identical adaptive search. (2) The measurement that co-tuning is qualitatively worse than a static exploitable judge: ∼4× the calibration gap, growing with search scale, and *lower* true quality of the selected winner—a mutual generator–judge Goodhart runaway. (3) The ablation showing independence from the search trace, not ensembling per se, is the active ingredient: an ensemble that shares the trace retains ∼13× the inflation of an independent one. (4) Pre-registered directional hypotheses, all holding in 24/24 independent rebuilds of the experiment. We deliberately do not claim the generate-and-select pattern, reward hacking, or ensemble mitigation as new; each is established [1, 2, 4]. The contribution is isolating *judge provenance* as the variable and measuring what co-tuning does.

2 RELATED WORK

Best-of-n reward hacking (static judge). Gao, Schulman, and Hilton [1] measure gold-vs-proxy divergence under optimization against a fixed reward model for *both* RL and best-of-n sampling, giving scaling laws for each; inference-time selection against a static proxy is therefore covered ground, and recent work proves rise-then-fall dynamics for the BoN class and offers mitigations [4]. Our fixed-judge condition A reproduces this known effect; our contribution begins where the judge stops being static. **Ensembles and their limits.** Coste et al. [2] show

reward-model ensembles largely eliminate BoN overoptimization; Eisenstein et al. [3] show the mitigation depends on ensemble members having *decorrelated* errors—correlated ("herding") ensembles underperform. We import that insight to inference-time judge ensembles and sharpen it: when all members are shaped by the generator's trace, ensembling retains an order of magnitude more inflation than independence (condition C vs D). **Generator–judge entanglement.** Pan et al. [5] observe spontaneous reward hacking in iterative self-refinement, with generator–evaluator *context sharing* increasing severity; preference-leakage work shows generator–judge *relatedness* (same model or lineage) inflates scores [6]. Both identify entanglement axes adjacent to ours; neither implements a judge refit on the search trace nor runs the provenance ablation. **LLM-as-judge reliability and judge gaming.** Zheng et al. [7] characterize judge biases under a fixed protocol; panels of diverse judges reduce single-judge bias [8]; deliberate attacks find a fixed judge's blind spots [9, 10]. Our setting differs: the exploitation is emergent from selection pressure plus co-tuning, not an engineered attack, and our fix criterion is independence from the trace rather than judge scale or diversity alone. **Selection inflation elsewhere.** The best-of-N statistic inflates under the null in finance [11, 12] and in noisy quality-diversity archives [13]; self-consistency-style pipelines reuse the generator's own signal for selection [14], which our result flags as a live risk whenever the critique signal is not independent of generation. Goodhart's law is the umbrella [15].

3 TESTBED AND DESIGN

Model. A candidate is a vector $c \in \mathbb{R}^K$, K=30; only the first M=3 coordinates carry true quality, $q(c) = \sum_{i \le M} c_i$; the remaining 27 are inert. A judge is a quirk vector u scoring $J_u(c) = q(c) + \langle c, u \rangle$: every judge sees true quality plus its own exploitable taste. Quirks are initialized i.i.d. $\mathcal{N}(0, 1)$ on the 27 inert coordinates and zero on the true coordinates; the ensembles in conditions C and D draw their 10 members independently the same way, and D's members are simply never updated afterward (their diversity is the independent initialization). There is no measurement noise anywhere; all inflation is selection and adaptation.

Adaptive search. The generator proposes in rounds (rounds = $\max(2, \min(6, \lfloor N/2 \rfloor))$, batch = $\lfloor N/\text{rounds} \rfloor$): proposals are $\mu + \mathcal{N}(0, 1)^K$, and after each round μ steps halfway toward the round winner. This makes the *search trace*—the sequence of batches and winners—a real object the judge can absorb.

Co-tuning. After each round a co-tuned judge *adds* η times the mean inert features of the round's top 20% to its quirk (u += $\eta\, \bar{c}_{\text{inert}}$, η=0.5)—a compounding accumulation, not a convex step, so the judge's taste both aligns with and *grows along* the directions the generator produces: the deterministic analogue of repeatedly appending the pipeline's own winners to a scoring rubric as exemplars. (An interpolating update u += $\eta(\bar{c} - u)$ yields the same directional results with smaller magnitudes—1.7× and 7.1× for the two headline ratios (Sec. 4)—so the accumulation is the aggressive case, not the enabling one.)

Conditions. Identical search structure; only judge provenance differs. **A** fixed single judge (static exploitable baseline, the case of [1]); **B** co-tuned single judge; **C** ensemble of 10 co-tuned judges, all refit on the same shared trace, selection on their mean; **D** ensemble of 10 judges never exposed to the trace, selection on their mean.

```
for round in 1..rounds:              # per condition
  C <- batch proposals ~ mu + N(0,1)^K
  s(c) <- mean_j [ q(c) + <c, u_j> ]  # judges J
  w <- argmax_C s;  keep best (w, s(w)) so far
  mu <- mu + 0.5 (w - mu)             # search adapts
  if co-tuned:                        # B and C only
    m <- mean inert features of top 20% of C by s
    for u_j in J:  u_j <- u_j + 0.5 m # accumulates
# report: best s seen; true quality: <w*, w_true>
```

Figure 1: The four conditions differ only in J: A one frozen u; B one co-tuned u; C ten co-tuned u_j sharing the trace; D ten frozen u_j.

Table 1: Provenance ablation at N=3,000 (seed-averaged over 24 reseeds): reported score of the final winner, its true quality, and the calibration gap (reported − true). All four pre-registered hypotheses hold in 100% of reseeds.

Condition	reported	true	inflation
A fixed single judge	58.4	5.6	52.8
B co-tuned single judge	215.0	3.5	**211.5**
C co-tuned ensemble (10)	163.4	6.4	156.9
D independent ensemble (10)	25.4	**13.4**	**12.0**

Metric. Each condition's *calibration gap* (inflation) is its own reported score of its final winner minus that winner's true quality—the additive gap a practitioner actually consumes. (We avoid ratio definitions: a held-out judge's score of another condition's winner is near zero by construction, making ratios ill-defined.) An untouched held-out judge is also logged. Deterministic and seeded; every number is written to `ablation_cotuned_results.json`; sweep $N \in \{10, \ldots, 3{,}000\}$, 24 independent reseeds × 12 runs.

Pre-registered hypotheses. H1: inflation(B)>inflation(A). H2: inflation(C)> 2×inflation(D). H3: D has the smallest inflation of all four. H4: true quality(D)≥true quality(B).

4 RESULTS

Baseline (static judge), for calibration. In the non-adaptive two-condition demo that preceded this ablation (same model, i.i.d. proposals, fixed judge), the winner's optimized-judge score climbs 8.5 → 19.5 as N goes 10 → 3,000 while true quality moves 0.9 → 2.0—an order-of-magnitude reported-vs-true gap at scale, with the additive inflation growing 7.8 → 17.7, and an independent-judge ensemble roughly doubling selected true quality (4.4 vs 2.0 at N=3,000); every directional claim holds in 24/24 reseeds. This reproduces known best-of-n inflation [1] in a zero-noise setting and sets the scale for what follows.

Co-tuning is a Goodhart runaway (H1). The co-tuned judge's gap is 211.5 vs the static judge's 52.8 at N=3,000 (4.0×; the ratio is at least 4× at every N and as high as ~6.5× at small N; directional pass rates recorded at N=3,000: 24/24 seeds) (Table 1, Fig. 2). The mechanism is mutual: the judge's quirk grows along

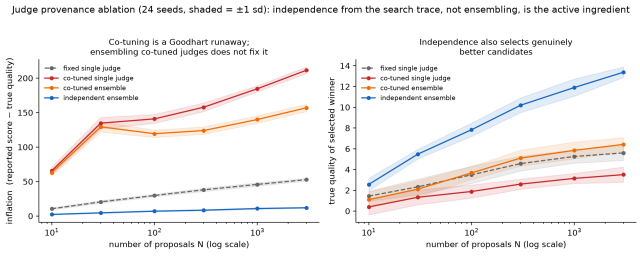

Figure 2: Judge-provenance ablation across N (24 seeds; bands ±1 sd). Left: the calibration gap under co-tuning (B, C) dwarfs the static judge (A) and keeps growing; the trace-independent ensemble (D) stays nearly flat. Right: D also selects winners of far higher *true* quality; the co-tuned judge (B) selects the worst.

Table 2: Calibration gap (reported − true) by condition across the sweep, seed-averaged over 24 reseeds (full columns incl. true quality and held-out scores in `ablation_cotuned_results.json`).

N	A fixed	B co-tuned	C co-t. ens.	D indep. ens.
10	10.5	65.6	62.9	2.3
30	20.5	134.7	129.3	4.6
100	29.8	140.9	119.4	7.0
300	38.0	157.8	124.0	8.4
1,000	45.8	184.6	140.0	10.8
3,000	52.8	211.5	156.9	12.0

the directions the generator concentrates in, the generator chases the enlarged quirk, and reported score compounds while true quality is left behind—indeed B's *true* quality (3.5) is the worst of all conditions, below even the static-judge baseline (5.6). A co-tuned judge is not a noisier judge; it is an accomplice. Part of B's reported-score growth is the judge's own scale growing (the accumulated quirk norm compounds)—that is the runaway mechanism itself, not a confound, but it means B's inflation is measured in units that inflate with it. The scale-free confirmation is true quality: no rescaling of a judge can make its *selections* genuinely worse, yet B selects the worst candidates of all four conditions.

Ensembling without independence fails (H2). The co-tuned ensemble retains a gap of 156.9—13.1× the independent ensemble's 12.0 at N=3,000, and the ratio is even larger at small N (~27×). Averaging judges whose errors were correlated by the shared trace cancels almost nothing; this is the inference-time analogue of ensemble herding [3], and it holds in 100% of seeds.

Independence is the active ingredient (H3, H4). The trace-independent ensemble is the only condition whose reported score stays close to truth (gap 2.3 → 12.0 across the whole sweep), and it simultaneously selects the genuinely best candidates: true quality 13.4 vs 5.6 (static) and 3.5 (co-tuned) at N=3,000. Independence pays twice—calibration and selection quality—and both margins hold in 24/24 seeds.

Sensitivity: directions are parameter-independent, multipliers are not. A 12-cell one-factor sweep around the baseline (co-tuning rate $\eta \in \{0.1, 0.25, 0.5, 1.0\}$; the *interpolating* update $u \mathrel{+}= \eta(\bar{c} - u)$ in place of accumulation; $K \in \{10, 30, 100\}$; ensemble size $\in \{3, 10, 30\}$; refit fraction $\in \{0.1, 0.2, 0.5\}$; 8 seeds × 6 runs per cell) finds all four pre-registered directions hold in 100% of seeds in *every* cell. The magnitudes move exactly as one should expect: the co-tuned/static inflation ratio ranges from 1.6× (gentlest, η=0.1) to 7.4× (η=1.0), and the co-tuned-ensemble/independent-ensemble ratio from 3.2× to 30.5×. The headline 4×/13× are therefore one point in a robust family, not the phenomenon itself; the phenomenon—co-tuning inflates beyond a static judge, shared-trace ensembling barely helps, independence closes the gap and selects better candidates—survives every parameterization tested, including the gentle one (`sensitivity_cotuned.py`).

Do cheap mitigations suffice? Two standard fixes, added as sweep cells, help but do not substitute for independence: *drift regularization* (shrinking the rubric halfway back to its initialization each round) leaves a gap of 81.1—1.8× the static judge, ~8× the independent ensemble—and *periodic judge refresh* every 2 rounds neutralizes the runaway only back to the static-judge floor (46.9 vs 45.5), which itself inflates ~4.5× the independent ensemble. Refresh *every* round does work (4.0)—but a judge replaced before it can absorb any trace is precisely a trace-independent judge: the mitigation succeeds exactly insofar as it reinstates independence, which is the paper's point.

5 DISCUSSION

For anyone building generate-and-select agents—strategy search, self-refining coders, automated red-teaming, prompt optimization—the operational rule is: *a judge that was tuned, prompted, or patched using the pipeline's own outputs is part of the attack surface, not ground truth.* "The judge said it was good" is closer to in-sample fit than to evaluation. The measured fix is cheap: hold out judges that have never seen the generator's trace, and aggregate across them; ensembling alone, without that independence, recovers little. The same risk statement covers benchmark–system co-evolution (leaderboards patched in response to the systems they rank) and self-refinement loops whose critique signal is the generator's own [5, 14].

Preliminary live instantiation. A micro-scale live run (LLM generator and LLM judges, one-line predictor expressions over five features with objective held-out R^2 as ground truth; 2 seeds × 3 rounds, all completions cached) reproduces the core phenomenon with a real judge: the selecting judge rates its final winner 8.0/10 while three held-out judges rate the same winner 5.3 (self-inflation gap +2.7), and the winners' *objective* quality is negative ($R^2 < 0$)—high reported scores on genuinely poor candidates. Selecting on the independent three-judge ensemble nearly closes the self-inflation gap (+0.3). At this scale the co-tuned and fixed single-judge conditions did not separate (identical trajectories); establishing the co-tuning amplification live requires a larger run and is the immediate next step. Script and cached completions: `llm_judge_instantiation.py`.

6 LIMITATIONS AND NEXT STEP

The testbed is synthetic and linear by design—provenance is the only manipulated variable, which is what makes the ablation clean, but effect *sizes* here are not predictions for any real system. Co-tuning is one operationalization (taste refit toward round winners); prompt-iteration on real LLM judges may be gentler or harsher. The preliminary live run above confirms single-judge self-inflation and the independent-ensemble fix with a real LLM judge, but is far too small to test the co-tuning amplification; scaling that live run (more rounds, more seeds, richer domains) is the immediate next step. The mechanism is evaluator-agnostic and the prediction is unambiguous.

REPRODUCIBILITY

Deterministic, dependency-free, seeded: `ablation_cotuned.py` (this ablation; 24 reseeds × 12 runs, directional pass rates emitted with the table) and `evaluator_overfit_demo.py` / `evaluator_overfit_robustness.py` (the static-judge baseline), each writing its results JSON alongside the script.

REFERENCES

[1] L. Gao, J. Schulman, and J. Hilton. 2023. Scaling Laws for Reward Model Overoptimization. *Proc. ICML*, PMLR 202, 10835–10866.

[2] T. Coste, U. Anwar, R. Kirk, and D. Krueger. 2024. Reward Model Ensembles Help Mitigate Overoptimization. *ICLR*. arXiv:2310.02743.

[3] J. Eisenstein et al. 2023. Helping or Herding? Reward Model Ensembles Mitigate but do not Eliminate Reward Hacking. arXiv:2312.09244.

[4] H. Khalaf, C. Mayrink Verdun, A. Oesterling, H. Lakkaraju, and F. du Pin Calmon. 2025. Inference-Time Reward Hacking in Large Language Models. *NeurIPS*. arXiv:2506.19248.

[5] J. Pan et al. 2024. Spontaneous Reward Hacking in Iterative Self-Refinement. arXiv:2407.04549.

[6] D. Li et al. 2025. Preference Leakage: A Contamination Problem in LLM-as-a-Judge. arXiv:2502.01534.

[7] L. Zheng et al. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. *NeurIPS Datasets and Benchmarks*.

[8] P. Verga et al. 2024. Replacing Judges with Juries: Evaluating LLM Generations with a Panel of Diverse Models. arXiv:2404.18796.

[9] X. Zheng, T. Pang, C. Du, Q. Liu, J. Jiang, and M. Lin. 2025. Cheating Automatic LLM Benchmarks: Null Models Achieve High Win Rates. *ICLR*. arXiv:2410.07137.

[10] J. Shi et al. 2024. Optimization-based Prompt Injection Attack to LLM-as-a-Judge. *Proc. CCS*. arXiv:2403.17710.

[11] D. H. Bailey and M. López de Prado. 2014. The Deflated Sharpe Ratio. *J. Portfolio Management* 40, 5, 94–107.

[12] R. Sullivan, A. Timmermann, and H. White. 1999. Data-Snooping, Technical Trading Rule Performance, and the Bootstrap. *J. Finance* 54, 5, 1647–1691.

[13] M. Flageat, J. Huber, F. Hélénon, S. Doncieux, and A. Cully. 2025. Extract-QD Framework: Quality-Diversity in Noisy, Stochastic or Uncertain Domains. *Proc. GECCO*.

[14] X. Wang et al. 2022. Self-Consistency Improves Chain of Thought Reasoning in Language Models. arXiv:2203.11171.

[15] C. A. E. Goodhart. 1975. Problems of Monetary Management: The U.K. Experience. *Papers in Monetary Economics*, Reserve Bank of Australia.